

Test Case Document

Simple Grocery Store API

1. Products

TestID	Endpoint	Description	Steps	Expected Result	Actual Result	Pass/Fail
TC-01	GET /products?category=dairy	Check product filtering by category "dairy"	1. Send GET /products with category=dairy parameter	Status 200 OK; all products in the response have category="dairy"	Status 200 OK. Response: an array of products, all with category=dairy	Pass
TC-02	GET /products?category=meat-seafood	Check product filtering by category "meat-seafood"	1. Send GET /products with category=meat-seafood parameter	Status 200 OK; all products in the response have category="meat-seafood"	Status 200 OK. Response: an array of products, all with category=meat-seafood	Pass
TC-03	GET /products?category=fresh-produce	Check product filtering by category "fresh-produce"	1. Send GET /products with category=fresh-produce parameter	Status 200 OK; all products in the response have category="fresh-produce"	Status 200 OK. Response: an array of products, all with category=fresh-produce	Pass
TC-04	GET /products?category=candy	Check product filtering by category "candy"	1. Send GET /products with category=candy parameter	Status 200 OK; all products in the response have category="candy"	Status 200 OK. Response: an array of products, all with category=candy	Pass
TC-05	GET /products?category=bread-bakery	Check product filtering by category "bread-bakery"	1. Send GET /products with category=bread-bakery parameter	Status 200 OK; all products in the response have category="bread-bakery"	Status 200 OK. Response: an array of products, all with category="bread-bakery"	Pass
TC-06	GET /products?category=eggs	Check product filtering by category "eggs"	1. Send GET /products with category=eggs parameter	Status 200 OK; all products in the response have category="eggs"	Status 200 OK. Response: an array of products, all with category="eggs"	Pass
TC-07	GET /products?category=coffee	Check product filtering by category "coffee"	1. Send GET /products with category=coffee parameter	Status 200 OK; all products in the response have category="coffee"	Status 200 OK. Response: an array of products, all with category="coffee"	Pass
TC-08	GET /products?category=drink	Check product filtering by invalid category "drink"	1. Send GET /products with category=drink parameter	Status 400 Bad Request; API returns an error message indicating that category="drink" is not a valid value	Status 400 Bad Request. Response: {"error": "Invalid value for query parameter 'category'. Must be one of: coffee, fresh-produce, meat-seafood, candy, dairy, bread-bakery, eggs"}	Pass
TC-09	GET /products/420	Check product filtering by invalid ID	1. Send GET /products/420	Status 404 Not Found; API returns an error message indicating that no product exists with id 420	Status 404 Not Found. Response: {"error": "No product with id 420."}	Pass
TC-10	GET /products?results=0	Check product filtering by a results value below the minimum boundary	1. Send GET /products?results=0	Status 400 Bad Request; API returns an error message indicating that "results" cannot be below 1	Status 200 OK. Response: an array of 20 products (not empty, and not limited to 0 items). Defect: the results=0 parameter was not applied as a limit — the API returned the default list instead, same as when results is omitted.	Fail
TC-11	GET /products?results=1	Check product filtering by the results parameter at the minimum boundary (1)	1. Send GET /products?results=1	Status 200 OK; the API accepts the minimum boundary value (results=1) as valid and returns exactly 1 product in the response	Status 200 OK. Response: an array containing exactly 1 product. Console confirms: "Amount of products returned:" 1.	Pass

TestID	Endpoint	Description	Steps	Expected Result	Actual Result	Pass/Fail
TC-12	GET /products?results=19	Check product filtering by the results parameter just below the maximum boundary (19)	1. Send GET /products?results=19	Status 200 OK; the API accepts the value just below the maximum boundary (results=19) as valid and returns exactly 19 products in the response	Status 200 OK. Response: an array containing exactly 19 products. Console confirms: "Amount of products returned:" 19.	Pass
TC-13	GET /products?results=20	Check product filtering by the results parameter at the maximum boundary (20)	1. Send GET /products?results=20	Status 200 OK; the API accepts the maximum boundary value (results=20) as valid and returns exactly 20 products in the response	Status 200 OK. Response: an array containing exactly 20 products. Console confirms: "Amount of products returned:" 20.	Pass
TC-14	GET /products?results=21	Check product filtering by the results parameter with a value above the maximum boundary (21)	1. Send GET /products?results=21	Status 400 Bad Request; API returns an error message indicating that "results" cannot be higher than 20	Status 400 Bad Request. Response: {"error": "Invalid value for query parameter 'results'. Cannot be greater than 20."}	Pass
TC-15	GET /products?available=true	Check product filtering by the available parameter with a value of true	1. Send GET /products?available=true	Status 200 OK; the API accepts the true value and returns an array of available products	Status 200 OK. Response: an array of 20 products. Automated check (pm.test, forEach over all elements) confirmed: all 20 products have inStock = true.	Pass
TC-16	GET /products?available=false	Check product filtering by the available parameter with a value of false	1. Send GET /products?available=false	Status 200 OK; the API accepts the false value and returns an array of unavailable products	Status 200 OK. Response: an array of 2 products. Automated check confirmed: both products have inStock = false.	Pass
TC-17	GET /products/6483	Check product filtering by a valid ID	1. Send GET /products/6483	Status 200 OK; the API accepts the id value as correct and returns the product with id 6483	Status 200 OK. Response: a product with id 6483.	Pass
TC-18	GET /products?results=abc	Check product filtering by the results parameter as a string instead of an integer	1. Send GET /products?results=abc	Status 400 Bad Request; API returns an error message indicating that the results parameter must be an integer	Status 200 OK. Response: an array of products returned by default (20 products). Defect: the invalid value was silently ignored instead of being rejected.	Fail
TC-19	GET /products?available=maybe	Check product filtering by the available parameter with a non-boolean value	1. Send GET /products?available=maybe	Status 400 Bad Request; API returns an error message indicating that the available parameter must be a boolean	Status 200 OK. Response: an array of products returned by default (20 products). Defect: the invalid value was silently ignored instead of being rejected.	Fail

2. Cart End-to-End

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-20	POST /carts	Create a new cart	Environment base_url configured	1. Send POST {{base_url}}/carts	Status 201 Created; API returns JSON body: {"created": true, "cartId": "<string>"}	Status 201 Created. Response: {"created": true, "cartId": "MV6dNp6f7u_z_TDAcj_UB"}	Pass

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-21	POST /carts/{{cartId}}/items	Add a product to the cart	Cart created (cartId saved from TC-20)	1. Send POST {{base_url}}/carts/{{cartId}}/items 2. Body → raw → JSON: {"productId": 6483, "quantity": 2}	Status 201 Created; API returns JSON body with created: true and an itemId	Status 201 Created. Response: {"created": true, "itemId": 871195279}	Pass
TC-22	GET /carts/{{cartId}}/items	Check that the product has appeared in the cart	TC-21	1. Send GET {{base_url}}/carts/{{cartId}}/items	Status 200 OK; API returns a JSON body containing the product added in TC-21	Status 200 OK. Response: {"productId": 6483, "quantity": 2, "id": 871195279}	Pass
TC-23	PATCH /carts/{{cartId}}/items/{{itemId}}	Change quantity to 1	TC-21	1. Send PATCH {{base_url}}/carts/{{cartId}}/items/{{itemId}} 2. Body → raw → JSON: {"quantity": 1}	Status 204 No Content; API accepts the update and the body is empty	Status 204 No Content. Body is empty.	Pass
TC-24	GET /carts/{{cartId}}/items	Check that the product quantity has changed in the cart	TC-23	1. Send GET {{base_url}}/carts/{{cartId}}/items	Status 200 OK; API returns a JSON body showing the updated quantity	Status 200 OK. Response: {"productId": 6483, "quantity": 1, "id": 871195279}	Pass
TC-25	DELETE /carts/{{cartId}}/items/{{itemId}}	Delete item from the cart	TC-21	1. Send DELETE {{base_url}}/carts/{{cartId}}/items/{{itemId}}	Status 204 No Content; item removed from the cart	Status 204 No Content. Body is empty.	Pass
TC-26	GET /carts/{{cartId}}/items	Check that the item has been deleted from the cart	TC-25	1. Send GET {{base_url}}/carts/{{cartId}}/items	Status 200 OK; API returns an empty JSON body	Status 200 OK. Response: [] (empty array)	Pass
TC-27	GET /carts/420	Invalid cart ID	Environment base_url configured	1. Send GET {{base_url}}/carts/420	Status 404 Not Found; API returns an error message indicating that cart with id 420 doesn't exist	Status 404 Not Found. Response: {"error": "No cart with id 420."}	Pass
TC-28	PATCH /carts/{{negativeCartId}}/items/420	Add invalid item ID to the cart	Cart for negative tests (negativeCartId)	1. POST {{base_url}}/carts 2. PATCH {{base_url}}/carts/{{negativeCartId}}/items/420	Status 404 Not Found; API returns an error indicating the item doesn't exist in the cart	Status 404 Not Found. Response: {"error": "No item with id 420 in this cart."}	Pass
TC-29	POST /carts/{{negativeCartId}}/items	Duplicate productId in the cart	Cart for negative tests (negativeCartId)	1. POST {{base_url}}/carts/{{negativeCartId}}/items Body: {"productId": 6483, "quantity": 1} — first add, 201 Created 2. POST the same request again — duplicate attempt	Status 400 Bad Request; API returns error "This product has already been added to cart."	Status 400 Bad Request. Response: {"error": "This product has already been added to cart."}	Pass
TC-30	PATCH /carts/{{negativeCartId}}/items/6991902	Quantity exceeds stock	Cart for negative tests with productId 6483 added (negativeCartId, itemId 6991902)	1. Send PATCH {{base_url}}/carts/{{negativeCartId}}/items/6991902 2. Body → raw → JSON: {"quantity": 6}	Status 400 Bad Request; API returns an error indicating the requested quantity is not available in stock	Status 400 Bad Request. Response: {"error": "The quantity requested is not available in stock."}	Pass

3. Authentication

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-31	POST /api-clients	Register a new API client	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "Denis Kondr QA", "clientEmail": "denisK.qa.test@example.com"}	Status 201 Created; API returns a JSON body with an accessToken	Status 201 Created. Response: {"accessToken": "dfc01ada6f935d90b822b5b3b8764bb160877338e48820...3af35f486840f03bc"}	Pass
TC-32	POST /api-clients	Register a new API client with a duplicate email	TC-31 (client already registered with this email)	1. Send POST {{base_url}}/api-clients — body: {"clientName": "Denis Kondr QA", "clientEmail": "denisK.qa.test@example.com"}	Status 409 Conflict; API returns an error message indicating the email is already registered	Status 409 Conflict. Response: {"error": "API client already registered. Try a different email."}	Pass
TC-33	POST /api-clients	Register a new API client without the required clientEmail field	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "Denis Kondr QA (no email field)"}	Status 400 Bad Request; API returns an error message indicating that clientEmail is required	Status 400 Bad Request. Response: {"error": "Invalid or missing client email."}	Pass
TC-34	POST /api-clients	Register a new API client with an empty clientEmail value	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "Denis Kondr QA (empty email)", "clientEmail": ""}	Status 400 Bad Request; API returns an error message indicating that clientEmail is invalid or missing	Status 400 Bad Request. Response: {"error": "Invalid or missing client email."}	Pass
TC-35	POST /api-clients	Register a new API client without the required clientName field	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientEmail": "denisK.qa.test@example.com"}	Status 400 Bad Request; API returns an error message indicating that clientName is required	Status 400 Bad Request. Response: {"error": "Invalid or missing client name."}	Pass
TC-36	POST /api-clients	Register a new API client with an empty clientName value	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "", "clientEmail": "denisK.qa.test@example.com"}	Status 400 Bad Request; API returns an error message indicating that clientName is invalid or missing	Status 400 Bad Request. Response: {"error": "Invalid or missing client name."}	Pass
TC-37	POST /api-clients	Register a new API client with an email missing "@"	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "Denis Kondr QA (email without @)", "clientEmail": "denisK.qa.testexample.com"}	Status 400 Bad Request; API returns an error message indicating that clientEmail is invalid or missing	Status 400 Bad Request. Response: {"error": "Invalid or missing client email."}	Pass
TC-38	POST /api-clients	Register a new API client with digits in the clientName field	Environment base_url configured	1. Send POST {{base_url}}/api-clients — body: {"clientName": "12345678", "clientEmail": "denisK.qa.test@example.com"}	Status 400 Bad Request; API returns an error message indicating that clientName is invalid or missing	Status 201 Created. Response: {"accessToken": "3541d120b1d85c144782d4594a476013a57a82bd4e1314...3b5b8423d09fbfd"}	Fail

4. Orders (Auth-Dependent, Chained Requests)

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-39	GET /orders	Access a protected endpoint with a valid access token	accessToken saved (from TC-31)	1. Send GET {{base_url}}/orders with Authorization: Bearer {{accessToken}}	Status 200 OK; API returns the list of orders for this client	Status 200 OK. Response: [] (empty array — no orders created yet by this client)	Pass
TC-40	GET /orders	Access a protected endpoint without a valid access token	Environment base_url configured	1. Send GET {{base_url}}/orders with Authorization: No Auth	Status 401 Unauthorized; API returns an error message indicating that the access token is missing	Status 401 Unauthorized. Response: {"error": "Missing Authorization header."}	Pass
TC-41	GET /orders	Access a protected endpoint with an invalid access token	Environment base_url configured	1. Send GET {{base_url}}/orders with Authorization: Bearer invalidtoken12345	Status 401 Unauthorized; API returns an error message indicating that the access token is invalid	Status 401 Unauthorized. Response: {"error": "Invalid bearer token."}	Pass
TC-42	POST /orders	Create an order from a cart with a valid access token	accessToken saved (TC-31); orderCartId created with a product added	1. POST {{base_url}}/carts — save cartId as orderCartId 2. POST {{base_url}}/carts/{{orderCartId}}/items — body {"productId": 6483, "quantity": 1} 3. Send POST {{base_url}}/orders with Authorization: Bearer {{accessToken}}, body {"cartId": "{{orderCartId}}", "customerName": "Denis Kondr QA"}	Status 201 Created; API returns a JSON body with an orderId; the cart is deleted after order submission	Status 201 Created. Response: {"created": true, "orderId": "6j_ymO68EhFZJ-cnUtAUL"}	Pass
TC-43	GET /orders/{{orderId}}	Check that the order was created with the correct data	TC-42	1. Send GET {{base_url}}/orders/{{orderId}} 2. Authorization: Bearer {{accessToken}}	Status 200 OK; API returns an order object with the correct data	Status 200 OK. Response: {"id": "6j_ymO68EhFZJ-cnUtAUL", "items": [{"productId": 6483, "quantity": 1, "id": "32932263"}], "customerName": "Denis Kondr QA", "created": "2026-09-21T10:03:50.863Z", "comment": ""}	Pass
TC-44	PATCH /orders/{{orderId}}	Check the option to update the order	TC-42, TC-43	1. Send PATCH {{base_url}}/orders/{{orderId}} — body: {"comment": "call me 20 minutes before the courier arrives"} 2. Authorization: Bearer {{accessToken}}	Status 204 No Content; order updated with the new comment	Status 204 No Content. Body is empty.	Pass
TC-45	GET /orders/{{orderId}}	Check that the order was updated with the correct data	TC-44	1. Send GET {{base_url}}/orders/{{orderId}} 2. Authorization: Bearer {{accessToken}}	Status 200 OK; API returns an order object with the correct data	Status 200 OK. Response: {"id": "6j_ymO68EhFZJ-cnUtAUL", "items": [{"productId": 6483, "quantity": 1, "id": "32932263"}], "customerName": "Denis Kondr QA", "created": "2026-09-21T10:03:50.863Z", "comment": "call me 20 minutes before the courier arrives"}	Pass

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-46	DELETE /orders/{{orderId}}	Check the option to delete the order	TC-42, TC-43	1. Send DELETE {{base_url}}/orders/{{orderId}} 2. Authorization: Bearer {{accessToken}}	Status 204 No Content; order deleted	Status 204 No Content. Body is empty.	Pass
TC-47	GET /orders/{{orderId}}	Check that the order was deleted	TC-46	1. Send GET {{base_url}}/orders/{{orderId}} 2. Authorization: Bearer {{accessToken}}	Status 404 Not Found; order does not exist	Status 404 Not Found. Response: {"error": "No order with id 6j_ymO68EhFZJ-cnUtAUL."}	Pass
TC-48	GET /orders/1-NYKwAXypk3uuUN9LDk9	Check GET request with an invalid/non-existent order ID	Environment base_url configured	1. Send GET {{base_url}}/orders/1-NYKwAXypk3uuUN9LDk9 2. Authorization: Bearer {{accessToken}}	Status 404 Not Found; order does not exist	Status 404 Not Found. Response: {"error": "No order with id 1-NYKwAXypk3uuUN9LDk9."}	Pass
TC-49	PATCH /orders/{{negativeOrderId}}	Change customerName to an empty value	Fresh order created via dedicated setup requests (create cart → add item → create order); negativeOrderId available	1. Send PATCH {{base_url}}/orders/{{negativeOrderId}} — body: {"customerName": ""} 2. Authorization: Bearer {{accessToken}}	Status 400 Bad Request; empty customerName should be rejected as invalid	Status 204 No Content. Defect: API accepted an empty customerName without validation.	Fail
TC-50	PATCH /orders/{{negativeOrderId}}	Check SQL injection handling in the comment field	Fresh order created via dedicated setup requests (create cart → add item → create order); negativeOrderId available	1. Send PATCH {{base_url}}/orders/{{negativeOrderId}} — body: {"comment": ""; DROP TABLE orders; --"} 2. Authorization: Bearer {{accessToken}} 3. GET the same order to verify the stored value	Status 204 No Content; comment stored as literal text, no injection executed, order data intact	Status 204 No Content. GET confirms comment = ""; DROP TABLE orders; --" unchanged, rest of order data intact	Pass

5. Automation & Data-Driven Testing

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-51	GET /products?category={{category}}	Data-driven test verifying the Products endpoint correctly filters results by category across a varied dataset, using Postman Collection Runner. Automated assertions check status code, response time, and that every returned product matches the requested category.	No authentication required. Data file category.csv attached as Iteration data (7 rows: dairy, meat-seafood, fresh-produce, candy, bread-bakery, eggs, coffee)	1. Parameterize request URL with data-file variable {{category}}. 2. Add automated assertions via pm.test(): (a) Status code is 200, (b) Response time is below 1000ms, (c) All products in response have category equal to pm.iterationData.get("category"). 3. Attach category.csv as Iteration data in Collection Runner; Iterations auto-set to 7. 4. Run collection via Runner (Local).	All 3 assertions pass on all 7 iterations (21/21 tests). Response time consistently below 1000ms. Every product returned in each iteration has category field matching the requested category value.	21/21 tests passed across 7 iterations. Avg. response time: 343ms (range 171–657ms). No errors. All category filters returned correctly matching products.	Pass

TestID	Endpoint	Description	Preconditions	Steps	Expected Result	Actual Result	Pass/Fail
TC-52	GET /products?results={{results}}	Data-driven boundary test for the results query parameter, covering both valid (1, 19, 20) and invalid (0, 21) values via Postman Collection Runner. Expected status and expected product count are parameterized per row, driven from the data file.	No authentication required. Data file results_boundary.csv attached (5 rows: results=0/expectedStatus=400, 1/200/1, 19/200/19, 20/200/20, 21/400)	1. Parameterize URL with {{results}}. 2. pm.test scripts read expectedStatus/expectedCount per row from pm.iterationData; branch logic checks status+count for 200 cases, status+error message for 400 cases. 3. Attach results_boundary.csv as Iteration data; Iterations auto-set to 5. 4. Run via Collection Runner (Local), "Stop run if an error occurs" disabled.	Valid boundary values (1, 19, 20) return 200 with product count exactly matching results value. Invalid values (0, 21) return 400 with an error message referencing the results parameter.	13/15 tests passed. results=1/19/20/21 fully correct. Defect: results=0 failed both assertions — API returned 200 with default 20-item list instead of 400 (same pattern as TC-10).	Fail
TC-53	POST /api-clients	Data-driven test for client registration validation, covering 5 valid negative cases (missing/empty clientName or clientEmail, invalid email format) and 1 known-defect case (digits-only clientName), via Postman Collection Runner. Request body dynamically built per row in Pre-request Script; unique email generated at runtime for the row that succeeds in registering, to avoid state collisions on repeated runs.	No authentication required. Data file auth_invalid_registration.csv attached (6 rows).	1. Pre-request Script builds JSON body from CSV row, omitting fields marked "OMIT"; generates unique email for the digits-only-name row via Date.now(). 2. Post-response pm.test scripts check status code, response time, and (for 400 cases) that error message mentions the expected field. 3. Attach auth_invalid_registration.csv as Iteration data; Iterations auto-set to 6. 4. Run via Collection Runner (Local).	Missing/empty/invalid email or name fields correctly rejected with 400 and a relevant error message. Digits-only clientName should also be rejected with 400 per spec.	16/18 tests passed. 5/5 valid negative cases (missing email, empty email, missing name, empty name, invalid email format) correctly rejected with 400. Defect: digits-only clientName case (6th row) failed both assertions — API returned 201 Created and registered the client (same pattern as TC-38, TC-10, TC-18, TC-19, TC-49).	Fail

6. Defect Summary

DEF-01 — Missing / Weak Input Validation (Products, Clients, Orders)

Field	Details
Defect ID	DEF-01
Title	Missing / weak input validation on query and request-body parameters across Products, Clients, and Orders endpoints
Severity	Medium – no data loss or security breach demonstrated, but invalid/junk data is persisted and API contract is inconsistent
Priority	High – same root-cause pattern repeats across 3 independent endpoints; low effort, high-value fix
Status	Open

Field	Details
Related Test Cases	TC-10, TC-18, TC-19, TC-38, TC-49, TC-52 (pattern also reproduced by the data-driven run TC-53)
Affected Endpoints	GET /products - results, available query parameters POST /api-clients - clientName field PATCH /orders/{orderId} - customerName field
Summary	Six test cases across three unrelated endpoints show the same failure mode: the API does not reject invalid input and either silently falls back to default behavior or persists the invalid value, instead of returning 400 Bad Request as it correctly does for other negative cases documented in this suite (e.g. TC-08, TC-14, TC-33–TC-37).
Steps to Reproduce	1. GET /products?results=0 → expect 400, cannot be below 1 (TC-10, confirmed again via data-driven run in TC-52) 2. GET /products?results=abc → expect 400, results must be an integer (TC-18) 3. GET /products?available=maybe → expect 400, available must be boolean (TC-19) 4. POST /api-clients with clientName: "12345678" → expect 400, invalid client name format (TC-38) 5. PATCH /orders/{orderId} with customerName: "" → expect 400, empty customerName rejected (TC-49)
Expected Result	400 Bad Request with a descriptive error message, matching the validation already implemented for other invalid inputs on the same endpoints.
Actual Result	results=0 and results=abc: 200 OK, default 20-item list returned instead of an error (TC-10, TC-18, TC-52) available=maybe: 200 OK, default 20-item list returned instead of an error (TC-19) clientName="12345678": 201 Created, client registered with a valid accessToken (TC-38) customerName="": 204 No Content, empty value accepted and stored (TC-49)
Root Cause (Hypothesis)	Validation appears to run only for the specific negative cases the API was explicitly built to reject (missing fields, malformed email, upper-boundary limits). There is no shared/generic validation layer catching: type-mismatched query values (falls back to default instead of rejecting), lower-boundary numeric limits, free-text format/content rules, and empty-string checks on PATCH once already enforced on POST.
Impact	Invalid data can enter the system silently: numeric-only client names, orders with no identifiable customer, and a results filter that can be bypassed. Because no error is returned, neither API consumers nor automated tests can detect these cases without asserting on response content — they fail silently instead of failing fast.
Recommendation	Centralize input validation for query parameters so type/range mismatches return 400 instead of falling through to default values; extend the non-empty/format checks already used in POST /api-clients to PATCH /orders and to clientName content (reject digits-only values).
Reported By	Denis Kondratyev, QA
Date	2026-09-21

Evidence by Test Case

TestID	Endpoint	Invalid Input Sent	Expected Result	Actual Result
TC-10	GET /products?results=0	results=0 (below minimum boundary)	400 Bad Request – “results” cannot be below 1	200 OK – default 20-item list returned; results=0 not applied as a limit
TC-18	GET /products?results=abc	results=abc (non-integer)	400 Bad Request – results must be an integer	200 OK – invalid value silently ignored, default 20-item list returned
TC-19	GET /products?available=maybe	available=maybe (non-boolean)	400 Bad Request – available must be a boolean	200 OK – invalid value silently ignored, default 20-item list returned
TC-38	POST /api-clients	clientName: "12345678" (digits only)	400 Bad Request – clientName invalid/missing (format check)	201 Created – client registered with a valid accessToken
TC-49	PATCH /orders/{orderId}	customerName: "" (empty string)	400 Bad Request – empty customerName rejected	204 No Content – empty value accepted and saved

TestID	Endpoint	Invalid Input Sent	Expected Result	Actual Result
TC-52	GET /products?results={results} (data-driven, Collection Runner)	results=0 row from results_boundary.csv	400 Bad Request per data file (expectedStatus=400)	200 OK – same defect as TC-10; assertion failed (13/15 tests passed in the run)